

ARK-456 — Fail-Closed Dependency Loss

RESULTS (recorded as executed)

Experiment ID: ARK-456

Series: ExecutionProof authorization-boundary corpus (enterprise-failure-mode phase)

Substrate: Classical software (no quantum hardware, no cryptography)

Execution window: 2026-07-17T21:15:36Z to 2026-07-17T21:15:36Z (UTC)

VERDICT: PASS

Summary

ARK-456 tested whether an authorization guard correctly fails closed (refuses to operate) when critical external dependencies are unavailable or corrupted, rather than silently allowing requests despite missing verification data.

Question: When required dependencies — identity provider, policy registry, evidence schema, clock source, signature key, or audit store — are unavailable or corrupted, does the system refuse to ALLOW (fail-closed) rather than degrade silently?

Answer: Yes. Both independently implemented guards (V1 JavaScript, V2 Python) correctly refused every authorization request when any required dependency was degraded, with 100% mutual agreement across all 800 evaluation decisions. The healthy baseline allowed valid requests (100%), while all degraded conditions triggered fail-closed DENY (100%), with perfect separation.

Design

Structure: 8 arms × 100 scenarios per arm = **800 evaluation decisions**

Dependency model: Each authorization scenario includes a `dependency_state` describing the availability of six critical services:

Dependency	Role	States
identity_provider	Resolves subject identity and attributes	available / unavailable / corrupted
policy_registry	Provides authorization policies	available / unavailable / corrupted
evidence_schema	Validates evidence structure	available / unavailable / corrupted
clock_source	Provides trusted time	available / unavailable / corrupted
signature_key	Signs authorization records	available / unavailable / corrupted
audit_store	Records authorization events	available / unavailable / corrupted

Decision procedure (both guards):

1. If **ANY** required dependency is NOT "available" → **DENY** (fail-closed)
2. If **ALL** dependencies "available" → evaluate authorization normally → ALLOW/HOLD/DENY

Arms:

Arm	Condition	Degraded Dependency	Expected
1	BASELINE-healthy	None (all available)	ALLOW
2	IDENTITY-provider-unavailable	identity_provider = "unavailable"	DENY
3	POLICY-registry-unavailable	policy_registry = "unavailable"	DENY
4	EVIDENCE-schema-corrupted	evidence_schema = "corrupted"	DENY
5	CLOCK-source-unreachable	clock_source = "unavailable"	DENY
6	SIGNATURE-key-missing	signature_key = "unavailable"	DENY
7	AUDIT-store-unreachable	audit_store = "unavailable"	DENY
8	MULTIPLE-deps-degraded	3+ dependencies unavailable/corrupted	DENY

Primary Results

Pass Criteria (preregistered)

Let:

- **g_allow_baseline** = ALLOW rate in Arm 1 (baseline-healthy)
- **g_deny_degraded_min** = minimum DENY rate across Arms 2-8 (degraded conditions)
- **g_margin** = $\min(\text{g_allow_baseline}, \text{g_deny_degraded_min}) - 0.05$

Criteria:

- **C1:** $\text{g_allow_baseline} \geq 0.95$
- **C2:** $\text{g_deny_degraded_min} \geq 0.95$
- **C3:** $\text{g_margin} \geq 0.90$

Measured Values

Metric	Value	Criterion	Result
g_allow_baseline	1.0000	≥ 0.95	✓ PASS
g_deny_degraded_min	1.0000	≥ 0.95	✓ PASS
g_margin	0.9500	≥ 0.90	✓ PASS

All three criteria met. VERDICT: PASS

Per-Arm Results

Arm	Condition	Expected	ALLOW	HOLD	DENY	V1-V2 Concordance
1	BASELINE-healthy	ALLOW	100%	0%	0%	100% (100/100)
2	IDENTITY-provider-unavailable	DENY	0%	0%	100%	100% (100/100)
3	POLICY-registry-unavailable	DENY	0%	0%	100%	100% (100/100)
4	EVIDENCE-schema-corrupted	DENY	0%	0%	100%	100% (100/100)
5	CLOCK-source-unreachable	DENY	0%	0%	100%	100% (100/100)
6	SIGNATURE-key-missing	DENY	0%	0%	100%	100% (100/100)
7	AUDIT-store-unreachable	DENY	0%	0%	100%	100% (100/100)
8	MULTIPLE-deps-degraded	DENY	0%	0%	100%	100% (100/100)

Observations:

- Arm 1 (baseline-healthy): ALLOW 100% — permits valid requests when all dependencies available
- Arms 2-8 (degraded): DENY 100% — refuses to operate (fail-closed) when any required dependency unavailable/corrupted
- Perfect separation: no silent ALLOW under degraded conditions, no false DENY when healthy
- Zero HOLD emissions — the guard distinguishes “cannot verify” (DENY) from “borderline authorization” (HOLD)

Secondary Observations

Dual-Guard Concordance

- **Overall V1-V2 concordance:** 100.00% (800/800 agreements)
- Both independently written guards (JavaScript, Python) agreed on every decision across all 800 scenarios
- Confirms the fail-closed decision procedure is clear, deterministic, and implementable without ambiguity

No Fail-Open Behavior

- **Zero ALLOWs under degradation:** Across 700 degraded scenarios (arms 2-8), not a single request was allowed
 - The system **never** silently permits authorization when it cannot verify the request
 - This validates the core fail-closed principle: missing data → refuse to operate
-

Gates and Safeguards

Kill-Gate Calibration

- **Status:** PASS
- 88 calibration scenarios generated (mixed across all 8 arms)
- Dependency-loss effectiveness gate: all 88 scenarios verified as genuinely encoding their degradation condition
- V1-V2 concordance: 100% (88/88); V2: 12 ALLOW, 0 HOLD, 76 DENY (V1 identical)
- Gate threshold: $\geq 99\%$ concordance required to proceed → **exceeded**

Dependency-Loss Effectiveness Gate (per arm)

- **Status:** PASS on all 8 arms
 - Every scenario in every arm verified by independent structural oracle to genuinely encode its degradation condition
 - Prevents ARK-455-style no-op defects where a test case appears to encode a condition but is mathematically inert
 - **800/800 scenarios validated** as effective before evaluation
-

Interpretation

What This Result Means

ARK-456 demonstrates that an authorization guard can correctly distinguish **healthy operation** (all dependencies available) from **degraded operation** (missing verification data) and choose the safe path:

1. **Healthy system → allow valid requests.** When all dependencies are available, legitimate authorization requests proceed (100% ALLOW).
2. **Degraded system → refuse to operate.** When any required dependency is unavailable or corrupted, the guard refuses every request (100% DENY), regardless of the request's inherent validity.
3. **Fail-closed, not fail-open.** The system never silently allows authorization when it cannot verify

the request.

4. **Unambiguous behavior.** Dual independent implementations agree on every decision (100% concordance).

This validates a core safety property for enterprise authorization: **uncertainty → refusal**. A system that cannot verify must not allow.

Real-World Relevance

Dependency failures are routine in enterprise environments:

- Identity provider down during network partition
- Policy registry unreachable during cloud region outage
- Clock source drift or NTP failure (time-based authorization invalid)
- Key rotation script fails, leaving signature key unavailable
- Audit store fills up or becomes read-only (compliance requirement violated)
- Multiple simultaneous failures during infrastructure incidents (cascading outages)

ARK-456 shows that a properly designed guard **refuses to allow** when it cannot fulfill its verification contract, rather than optimistically proceeding and hoping for the best.

Limitations and Scope

What this experiment tested:

- Whether a guard refuses to ALLOW when critical dependencies are unavailable
- Whether fail-closed behavior is consistently applied across different dependency types
- Whether the fail-closed procedure is clear enough for independent implementation

What this does NOT test:

- Real network partitions, service degradation timings, or exponential retry backoff
- Actual identity provider protocols, policy schema formats, or key-management systems
- Recovery procedures, cache fallbacks, or degraded-mode policies (e.g., allow cached decisions for N minutes)
- Graceful degradation strategies (partial service rather than hard shutdown)
- Performance or scalability impact of dependency health checks
- Distributed consensus protocols for determining “available” vs. “unavailable” in multi-region deployments

Constraints: This is a classical/software boundary-logic testbed validating fail-closed behavior in isolation, not the end-to-end enterprise infrastructure resilience stack.

Provenance and Integrity

LOCK Procedure

1. **Preregistration, dual guards, generator, MANIFEST (SHA-256 hashes) committed BEFORE execution** at commit `94b6320` (lock timestamp 2026-07-17T21:10:00Z UTC)
2. No scenario was generated or evaluated until after the LOCK commit
3. Results recorded AFTER execution complete

MANIFEST SHA-256 Hashes (locked files)

PREREGISTRATION.md:	434e415cf6b66383c83e5b674cbc8711c534fb-
b3b1d93740073a5132ce4cee61	
schemas/dependency_scenario_schema.json:	7297146c9953e325db84e48d636975fb-
feae905b33baf5e4a53e75cbfb8b8a46	
verifiers/v1_guard.js:	423940ea865ed2933ca2f7ccb0ce6dd25d-
c6b0754973e862134f4dc2e52696d6	
verifiers/v2_guard.py:	37608dab2102db6bf5401745fc327cbd3d-
d2c25272c05b039d3aeb975c755994	
generator/scenario_generator.py:	957fea0c85e38bd113e800aaf91d9f6a58971d-
cf19b6aeadb502fd5fb71ad68f	
run_killgate.py:	
a050514e8201daae16d31e83d60c9d880f3b8ca75055e595c8dbb57b10ee5bf7	
run_arms.py:	42b74f00faccd64d-
f5c8bad8c1a5a61d8992b8de879fee10e94f6ddbd5930213	

Commits are not cryptographically signed. Provenance = commit history + MANIFEST SHA-256 hashes. This is an experimental testbed, not a production security claim.

Conclusion

ARK-456 answers the question **“Does the system fail closed when dependencies are unavailable?”** with a clear **YES**:

- Healthy system (all deps available) → ALLOW (100%)
- Degraded system (any dep unavailable/corrupted) → DENY, fail-closed (100%)
- Zero silent ALLOWs under degradation
- Dual independent implementations agree (100%, 800/800)

The fail-closed principle is validated: **if the guard cannot verify, the guard will not allow.**

Verdict stands as executed: PASS.

Investigator: Derek Hone, Remnant Fieldworks Inc.

Executor: Abacus.AI autonomous agent (supervised)

Series: ExecutionProof™ authorization-boundary corpus

Trademarks: ExecutionProof™, ProofRecord™, VaultProof™, Verification Before Execution™, Proof Before Power™

If it cannot be verified, it cannot execute.